

NIELIT O LEVEL • UNIT 06

JavaScript and Angular JS

Curriculum Hours: 25h (10h Theory + 15h Lab) | Topics: 11 | Language: English

UNIT OVERVIEW & PEDAGOGICAL SCOPE

JavaScript is the universal, high-level, just-in-time compiled client-side programming language of the World Wide Web, while AngularJS is a structural Model-View-Controller (MVC) front-end framework developed by Google for building dynamic Single Page Applications (SPAs). Together, they transform static HTML/CSS web pages into interactive, dynamic software applications. This unit teaches client-side scripting principles, JavaScript variable scopes (`var`, `let`, `const`), operators and strict equality (`===`), conditional logic (`if-else`, `switch`), popup dialog boxes (`alert`, `confirm`, `prompt`), DOM event listeners (`onclick`, `onchange`, `onsubmit`), client-side form validation mechanisms, and AngularJS MVC architecture, expressions `{{ }}`, modules, controllers, and core directives (`ng-app`, `ng-model`, `ng-bind`, `ng-repeat`, `ng-init`).

WHAT YOU WILL MASTER IN THIS UNIT

- Client-side scripting principles and the historical distinction between JavaScript and Sun Java.
- Variable declaration paradigms: `var` (function-scoped), `let` (block-scoped), and `const` (immutable identifier).
- JavaScript operators: Arithmetic, Comparison (type coercion `==` vs strict equality `===`), and Logical (`&&`, `||`, `!`).
- Conditional decision flow: `if`, `if...else`, multi-branch `if...else if`, and `switch...case` statements.
- Popup dialog boxes: `alert()` (notification), `confirm()` (boolean true/false confirmation), and `prompt()` (string input).
- DOM Event handling: `onclick`, `onchange`, `onsubmit`, `onload`, `onmouseover`, and `onmouseout`.
- Client-side form validation techniques: Empty checks, password length, and `return false` form submission cancellation.
- AngularJS framework architecture: Single Page Applications (SPAs) and Model-View-Controller (MVC) separation of concerns.
- AngularJS Expressions: Double curly braces `{{ expression }}` for evaluated outputs.
- AngularJS Modules & Controllers: `angular.module('myApp', [])` and `app.controller('myCtrl', function($scope){})`.
- Core AngularJS Directives: `ng-app`, `ng-model` (two-way binding), `ng-bind`, `ng-repeat`, and `ng-init`.

TOPIC 6.1: INTRODUCTION TO CLIENT SIDE SCRIPTING LANGUAGE

JavaScript is an interpreted, lightweight, object-based client-side programming language that executes inside the user's web browser engine to create dynamic and interactive web pages.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

HTML provides static structure and CSS provides visual styling, but only JavaScript can respond to user clicks, calculate totals, validate form data, animate menus, and fetch live data without reloading pages.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

Web browsers feature built-in JavaScript runtime engines (e.g. Google V8 in Chrome/Edge, SpiderMonkey in Firefox) that parse, compile, and execute scripts in a secure browser sandbox.

CRITICAL DISTINCTION: JAVASCRIPT VS. JAVA

Attribute / Parameter	JavaScript (Client-Side Web Scripting)	Java (General Purpose OOP Language)
-----------------------	--	-------------------------------------

Creator & Origin	Created by Brendan Eich in 1995 at Netscape (originally Mocha/LiveScript)	Created by James Gosling in 1995 at Sun Microsystems
Execution Environment	Runs inside web browsers (Chrome, Firefox, Safari, Edge)	Runs on Java Virtual Machine (JVM); requires compiled <code>.class</code> bytecode
Language Paradigm	Interpreted / JIT-compiled dynamic scripting language	Compiled, strongly typed, class-based object-oriented language
Typing Nature	Loosely Typed / Dynamic (<code>let x = 10; x = 'hello';</code> is valid)	Strictly / Statically Typed (<code>int x = 10;</code> cannot become string)
Primary Use Case	Client-side web interactivity, DOM manipulation, form validation	Enterprise backend software, Android mobile apps, desktop systems

When you click 'Show Password' on a login screen: A 2-line JavaScript script catches the click event and changes `<input type="password">` to `<input type="text">`. The masked dots instantly turn into readable letters without contacting the server.

- JavaScript was created by Brendan Eich in 1995 at Netscape.
- JavaScript is NOT related to Java; they are two completely different languages.
- JavaScript is loosely typed and executed directly inside client web browsers.
- JavaScript is embedded in HTML using the `<script>` tag.

QUICK TOPIC RECAP

JavaScript (created by Brendan Eich in 1995) is the client-side programming language of the web. It is dynamically typed, executed by browser engines, and unrelated to Java.

TOPIC 6.2: VARIABLES IN JAVASCRIPT (VAR, LET, CONST)

A variable is a named storage container in computer memory used to hold data values that can be referenced, manipulated, and modified throughout a program.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Web applications need to track user inputs, shopping cart totals, login states, and quiz scores. Variables store this dynamic state in memory.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

JavaScript provides three keywords to declare variables: `var` (traditional ES5), `let` (modern ES6 block-scoped), and `const` (modern ES6 constant).

HTML/CSS/JS

```
// Variable declarations in JavaScript
var score = 50; // Function-scoped (legacy)
let studentAge = 21; // Block-scoped (value can change)
const PI = 3.14159; // Constant (cannot be reassigned)

studentAge = 22; // Valid
// PI = 3.14; // TypeError: Assignment to constant variable

if (true) {
  let blockVar = "Visible only inside this if-block";
}
// console.log(blockVar); // ReferenceError: blockVar is not defined
```

JAVASCRIPT DATA TYPES

- Primitive Types: `Number` (integers and floats), `String` ('text'), `Boolean` ('true'/'false'), `Undefined` (unassigned variable), `Null` (intentional empty value), `Symbol`, `BigInt`.
- Non-Primitive Types: `Object`, `Array` (ordered list `[1, 2, 3]`), `Function`.
- Dynamic Typing: The `typeof` operator inspects the active data type (e.g. `typeof 42` returns `number`).

VAR VS. LET VS. CONST COMPARISON

Keyword	Scope Level	Re-declaration	Re-assignment	Hoisting Behavior
<code>var</code>	Function Scope (or Global)	Allowed (can re-declare same variable name)	Allowed (value can change)	Hoisted with `undefined` value
<code>let</code>	Block Scope `{ }`	Forbidden in same block	Allowed (value can change)	Hoisted in Temporal Dead Zone
<code>const</code>	Block Scope `{ }`	Forbidden in same block	Forbidden (immutable identifier)	Hoisted in Temporal Dead Zone

When calculating exam percentages: Declare `const totalMarks = 500;` (because total marks never change). Declare `let studentMarks = 420;` (because individual marks vary).

- `let` and `const` were introduced in ECMAScript 6 (ES6 / 2015) to fix `var` scoping bugs.
- `let` is block-scoped and allows value reassignment.
- `const` is block-scoped and cannot be reassigned after declaration.
- JavaScript is dynamically typed: variables do not require explicit type declarations.

QUICK TOPIC RECAP

Variables: `var` (function scope, legacy), `let` (block scope, reassignable), and `const` (block scope, immutable constant). Primitive types: number, string, boolean, undefined, null.

TOPIC 6.3: OPERATORS IN JS

An operator is a special programming symbol that performs mathematical, comparative, or logical operations on one or more operands (values or variables).

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Operators drive all computational logic in web programs, from calculating discounts and shopping totals to verifying login passwords.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

Operators evaluate expressions according to strict precedence rules (e.g. multiplication occurs before addition) and return a calculated result.

HTML/CSS/JS

```
// Loose vs Strict Equality in JavaScript
console.log(5 == "5"); // true (Number 5 coerced to String "5")
console.log(5 === "5"); // false (Number is NOT identical to String)

console.log(0 == false); // true
console.log(0 === false); // false

// Logical Operators
let isStudent = true;
let hasIDCard = true;
if (isStudent && hasIDCard) {
  console.log("Discount Applied!");
}
```

THE CRITICAL DISTINCTION: == VS. ===

- `==` (Abstract / Loose Equality): Compares value ONLY after performing automatic type coercion (e.g. `5 == '5'` evaluates to `true`).
- `===` (Strict Equality): Compares BOTH **Value AND Data Type** without coercion (e.g. `5 === '5'` evaluates to `false` because number `!=` string).
- Best Practice: Always use `===` and `!==` in modern JavaScript to eliminate unexpected type-conversion bugs.

CORE JAVASCRIPT OPERATOR CATEGORIES

Category	Operator Symbols	Operation Description	Example Expression & Result
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>**</code>	Mathematical calculations (<code>%</code> is Modulus remainder, <code>**</code> is Exponent)	<code>10 % 3</code> returns <code>1</code> (remainder)
Assignment	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code>	Assigns or modifies variable values	<code>x += 5</code> is shorthand for <code>x = x + 5</code>
Comparison	<code>==</code> , <code>===</code> , <code>!=</code> , <code>!==</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>	Compares two operands and returns a Boolean (<code>true</code> / <code>false</code>)	<code>5 === '5'</code> returns <code>false</code> (type mismatch)
Logical	<code>&&</code> (AND), <code> </code> (OR), <code>!</code> (NOT)	Combines Boolean conditions for decision logic	<code>(age >= 18 && feePaid)</code>
Ternary	<code>condition ? expr1 : expr2</code>	Compact conditional expression shorthand	<code>age >= 18 ? 'Adult' : 'Minor'</code>

In an exam portal: A student enters their passing score as string `"50"`. If you check `score === 50`, it returns `false` (type mismatch). Using `parseInt(score) === 50` guarantees type safety.

- `%` is the modulus operator that returns the remainder of a division.
- `==` checks value equality with type conversion; `===` checks strict value AND data type equality.
- `&&` returns true only if BOTH operands are true; `||` returns true if AT LEAST ONE is true.

QUICK TOPIC RECAP

Operators: Arithmetic (`+`, `-`, `*`, `/`, `%` remainder), Comparison (`==` loose vs `===` strict value+type), Logical (`&&` AND, `||` OR, `!` NOT), and Ternary (`? :`).

TOPIC 6.4: CONDITIONS STATEMENTS

Conditional statements are control flow structures that execute different blocks of code based on whether a specified Boolean test condition evaluates to `true` or `false`.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Conditional statements allow programs to make decisions: deciding whether to grant user access, display passing/failing grades, or show promotional discounts.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

JavaScript evaluates conditions from top to bottom. As soon as a branch tests `true`, its block executes and remaining branches are bypassed.

HTML/CSS/JS

```
// 1. if...else if...else Multi-Branch Decision
let marks = 82;
let grade;

if (marks >= 85) {
  grade = "S (Super)";
} else if (marks >= 75) {
  grade = "A (Excellent)";
} else if (marks >= 65) {
  grade = "B (Good)";
} else if (marks >= 55) {
  grade = "C (Fair)";
} else if (marks >= 50) {
  grade = "D (Pass)";
} else {
  grade = "F (Fail)";
}

// 2. switch...case Statement
let dayNumber = 3;
let dayName;

switch (dayNumber) {
  case 1: dayName = "Monday"; break;
  case 2: dayName = "Tuesday"; break;
  case 3: dayName = "Wednesday"; break;
  default: dayName = "Invalid Day";
}
```

IF-ELSE VS. SWITCH-CASE COMPARISON

Evaluation Parameter	`if...else if...else` Structure	`switch(expression)...case` Structure
Condition Types	Evaluates complex Boolean logical expressions (`<`, `>`, `&&`, `  `)	Evaluates a single expression against fixed constant values (`case value:`)
Data Types Tested	Tests any data type, ranges, and compound mathematical conditions	Tests discrete integers, strings, or enums
Execution Flow	Checks conditions sequentially one by one from top to bottom	Jumps directly to matching case; requires `break;` to prevent fall-through
Default Fallback	Handled by the optional closing `else { ... }` block	Handled by the `default: { ... }` label

Determining O-Level exam eligibility: Write `if (attendance >= 75 && feeCleared === true) { allowExam(); } else { blockHallTicket(); }`. The system makes an automated decision based on student status.

- `if` executes code only when the test condition evaluates to `true`.
- In a `switch` statement, the `break;` keyword is mandatory to prevent execution from 'falling through' to the next case.
- `default:` in a switch statement acts like the final fallback `else` block.

QUICK TOPIC RECAP

Conditional flow: `if...else` handles logical and range decisions. `switch...case` handles discrete value matching with mandatory `break;` statements.

TOPIC 6.5: JS POPUP BOXES (ALERT, CONFIRM, PROMPT)

JavaScript popup boxes are modal browser dialog windows built into the Window object (`window.alert``, `window.confirm``, `window.prompt``) that pause code execution until the user interacts.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Popup boxes provide instant ways to display urgent warnings, solicit user confirmation before permanent deletions, and accept quick textual inputs.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

When invoked, the browser pauses JavaScript execution and locks the webpage. The user must click a button before the webpage resumes.

HTML/CSS/JS

```
// 1. Alert box
alert("Exam registration submitted successfully!");

// 2. Confirm box
let userChoice = confirm("Are you sure you want to delete this record?");
if (userChoice === true) {
  console.log("Record deleted permanently.");
} else {
  console.log("Deletion canceled.");
}

// 3. Prompt box
let studentName = prompt("Please enter your full name:", "Guest");
if (studentName !== null) {
  alert("Welcome to O-Level Studio, " + studentName);
}
```

THE 3 JAVASCRIPT POPUP BOXES COMPARED

Popup Method	Visual Dialog Description	Buttons Present	Return Value to JavaScript
<code>alert(message)`</code>	Displays an informational message or warning dialog	Single [OK] button only	Returns <code>undefined`</code> (used purely for notification)
<code>confirm(message)`</code>	Asks a verification question requiring user consent	Two buttons: [OK] and [Cancel]	Returns true`</b> (if OK clicked) or <b>false` (if Cancel clicked)
<code>prompt(text, default)`</code>	Displays a message along with a single-line text input field	Two buttons: [OK] and [Cancel]	Returns Typed String`</b> (if OK) or <b>null` (if Cancel clicked)

Before permanently deleting a student record: Trigger `if (confirm("Are you sure you want to delete?")) { deleteRecord(); }`. If the admin clicks 'Cancel', the function aborts and the record is safe.

- `alert()` displays a notification with an OK button and returns nothing.
- `confirm()` displays OK and Cancel, returning Boolean `true`` or `false``.
- `prompt()` displays a text box, returning the entered string or `null`` if cancelled.
- Popup boxes are modal: they halt browser thread execution until dismissed.

QUICK TOPIC RECAP

Popup dialogs: `alert()` (notify, OK button), `confirm()` (verify, returns true/false), `prompt()` (inputs text, returns string or null).

TOPIC 6.6: JS EVENTS

A JavaScript Event is an action or occurrence detected by the browser-such as a mouse click, key press, page load, or form change-to which program code can respond.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Events are the foundation of interactive web development. Without events, web pages would be completely passive and incapable of reacting to user actions.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

Developers attach **Event Handlers / Listeners** to HTML elements. When the specified event occurs, the browser invokes the linked JavaScript function.

HTML/CSS/JS

```
<!-- Inline Event Handlers Example -->
<button onclick="changeBackground()">Change Page Color</button>

<input type="text"
onfocus="this.style.background='yellow'"
onblur="this.style.background='white'">

<script>
function changeBackground() {
document.body.style.backgroundColor = "#e0f2fe";
}
</script>
```

CORE HTML / JAVASCRIPT DOM EVENTS

Event Name	Triggering User / Browser Action	Practical Web Developer Use Case
`onclick`	User clicks the mouse button on an HTML element	Submitting forms, toggling navigation menus, opening modals
`onchange`	Value of an input field or dropdown selection changes	Updating state dropdown when a country is selected
`onsubmit`	User submits a ` <form>` (clicks submit button or presses Enter)</form>	Intercepting form to validate inputs before network transmission
`onload`	The entire webpage, including images and stylesheets, finishes loading	Initializing web applications, displaying welcome dialogs
`onmouseover`	Mouse cursor enters the bounding box of an element	Displaying tooltips, expanding sub-menus, image hover effects
`onmouseout`	Mouse cursor leaves the bounding box of an element	Hiding tooltips, collapsing sub-menus, restoring default colors
`onfocus` / `onblur`	Input field gains or loses active cursor focus	Highlighting active form inputs / validating field on exit

When a user types into an 'Age' input field and clicks away (`onblur`): The `onblur` event immediately triggers a check. If age < 18, it displays a red warning before the user even finishes the rest of the form.

- `onclick` triggers when an element is clicked.
- `onsubmit` is attached to the `` element to intercept and validate form submissions.
- `onload` triggers when the DOM and all page assets have finished loading.
- `onmouseover` and `onmouseout` respond to mouse movement.

QUICK TOPIC RECAP

Events respond to user actions: `onclick` (clicks), `onchange` (value changes), `onsubmit` (form send), `onload` (page ready), `onmouseover`/`onmouseout` (mouse hover).

TOPIC 6.7: BASIC FORM VALIDATIONS IN JAVASCRIPT

Client-side form validation is the programmatic verification of user inputs (checking for empty fields, password lengths, email syntax, numeric constraints) before transmitting data to the server.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Validating data on the client side provides instantaneous feedback to users, prevents corrupted database entries, and saves server bandwidth by blocking invalid submissions.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

The form's `onsubmit` event calls a JavaScript validation function. If errors are detected, the function alerts the user and executes `**return false;**`, which instructs the browser to cancel form submission.

HTML/CSS/JS

```
<!-- Complete JavaScript Form Validation Example -->
<form name="regForm" action="register.php" onsubmit="return validateForm()" method="POST">
<label>Student Name:</label>
<input type="text" name="fname"><br><br>

<label>Password (min 6 chars):</label>
<input type="password" name="fpass"><br><br>

<input type="submit" value="Register">
</form>

<script>
function validateForm() {
// 1. Read input values using DOM
let name = document.forms["regForm"]["fname"].value.trim();
let pass = document.forms["regForm"]["fpass"].value;

// 2. Validate empty name
if (name === "") {
alert("Error: Student Name cannot be left blank!");
return false; // Cancels form submission
}

// 3. Validate password length
if (pass.length < 6) {
alert("Error: Password must contain at least 6 characters!");
return false; // Cancels form submission
}

// All checks passed
return true; // Allows form submission to register.php
}
</script>
```

THE GOLDEN RULE OF CLIENT-SIDE VALIDATION

- ``return false;`` Cancellation: If validation fails, returning `false` inside the ``onsubmit="return validateForm()"`` attribute stops the browser from submitting the form.
- DOM Form Access: Access inputs via ``document.forms["formName"]["inputName"].value`` or ``document.getElementById("id").value``.
- String Trimming: Always run ``.trim()`` on text inputs to prevent users from bypassing required fields by typing empty spacebar characters.

- Client-Side vs Server-Side Validation: Client-side validation improves user experience, but **Server-Side validation is still mandatory for security** because malicious users can disable browser JavaScript.

A user tries to submit an exam application without entering their roll number: The `validateForm()` script catches the empty string, displays `alert("Roll Number is required!")`, focuses the cursor on the roll number input, and returns `false`. The page does not reload.

- Form validation is attached to the `<form>` element using `onsubmit="return validateForm()"`.
- Returning `false` cancels the form submission; returning `true` permits submission.
- Client-side validation provides fast user feedback, but server-side validation is still required for security.

QUICK TOPIC RECAP

JavaScript form validation intercepts `onsubmit`. If inputs are invalid, alert the error and `return false;` to cancel submission. Return `true;` to proceed.

TOPIC 6.8: INTRODUCTION TO ANGULAR JS

AngularJS is an open-source, structural front-end JavaScript framework developed by Google (Misko Hevery, 2009) designed for building dynamic **Single Page Applications (SPAs)** using an extended HTML syntax.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Traditional web apps require full-page reloads for every click. AngularJS extends HTML with custom attributes, providing two-way data binding and seamless desktop-app-like user experiences without page refreshes.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

AngularJS implements the **Model-View-Controller (MVC)** architectural design pattern, completely separating business data (Model) from visual HTML presentation (View) and operational logic (Controller).

WHAT IS A SINGLE PAGE APPLICATION (SPA)?

- Definition:** A web application that loads a single HTML shell page (`index.html`) once. Subsequent page changes dynamically update parts of the DOM without full page reloads.
- Two-Way Data Binding:** Any change in the UI input automatically updates the Model in memory, and any change in the Model instantly updates the UI in real time.
- Google Heritage:** Maintained and released by Google; uses the `angular.js` script library.

ANGULARJS MVC ARCHITECTURE BREAKDOWN

MVC Component	Role in AngularJS Applications	Implementation Technology
Model	The application data state and variables (e.g. user details, cart items)	JavaScript properties stored on the <code>\$scope</code> object
View	The visual HTML template rendered in the browser window	HTML decorated with AngularJS directives (<code>ng-app</code> , <code>ng-model</code> , <code>{{ }}</code>)
Controller	The business logic function that bridges the Model and the View	JavaScript functions linked via <code>ng-controller</code>

In Gmail (a classic SPA): Clicking an email in your inbox instantly opens the message in the center panel without the entire page flashing white or reloading. This seamless behavior is powered by SPA architecture.

- AngularJS was created by Misko Hevery in 2009 and is maintained by Google.
- AngularJS is built upon the Model-View-Controller (MVC) architectural design pattern.
- AngularJS powers Single Page Applications (SPAs) with two-way data binding.

QUICK TOPIC RECAP

AngularJS is a Google front-end JS framework for building Single Page Applications (SPAs) using MVC architecture and real-time two-way data binding.

TOPIC 6.9: ANGULAR JS EXPRESSIONS

AngularJS Expressions are JavaScript-like code snippets written inside **double curly braces** `{{ expression }}` that evaluate data and output the result directly into the HTML View.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Expressions allow web developers to interpolate dynamic variables, calculate mathematical formulas, format strings, and evaluate conditions directly in HTML without writing manual DOM manipulation code.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

The AngularJS compiler parses the double curly braces, evaluates the enclosed expression against the active ``$scope`` context, and writes the output into the DOM.

HTML/CSS/JS

```
<!-- Demonstrating AngularJS Expressions -->
<div ng-app="">
  <!-- Mathematical Calculation -->
  <p>5 + 10 = {{ 5 + 10 }}</p>

  <!-- String Concatenation -->
  <p>Greeting: {{ "Welcome, " + "Saurabh Tiwari" }}</p>

  <!-- Evaluating Scope Variables -->
  <p ng-init="quantity=5; cost=20">
    Total Bill: {{ quantity * cost }}
  </p>
</div>
```

ANGULARJS EXPRESSIONS VS. STANDARD JAVASCRIPT

Feature / Parameter	AngularJS Expressions <code>{{ }}</code>	Standard JavaScript Code
Enclosing Syntax	Enclosed in double curly braces: <code>{{ expression }}</code>	Written inside <code><script></code> tags or JS files
Error Handling	Forgiving: <code>null</code> or <code>undefined</code> outputs empty string (No error)	Strict: accessing properties of undefined throws <code>TypeError</code>
Control Flow	Does NOT support loops (<code>for</code>), conditions (<code>if</code>), or blocks	Supports full programming logic, loops, branches, functions
Filters Support	Supports formatting filters: <code>{{ cost currency }}</code>	Requires custom formatting functions

In a shopping cart: Write `<p>Total Cost: {{ quantity * price }}</p>`. As soon as a customer types ``3`` into quantity, the bill immediately displays ``150`` automatically with zero event listeners.

- AngularJS expressions are written inside double curly braces: `{{ expression }}`.
- Expressions can contain numbers, strings, operators, and `{{\$scope}}` variables.
- Unlike standard JavaScript, AngularJS expressions are forgiving and never throw crash errors if a variable is undefined.

QUICK TOPIC RECAP

AngularJS expressions (`{{ expression }}`) evaluate math, strings, and scope data inside HTML. They output results cleanly without throwing crash errors.

TOPIC 6.10: ANGULAR JS MODULES

An AngularJS Module defines an application container. It is the central registry where controllers, directives, services, and configuration logic are registered and organized.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Without modules, application code would pollute the global namespace. Modules keep application code clean, modular, maintainable, and testable.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

A module is created in JavaScript using `angular.module('moduleName', [])`. It is linked to HTML using the `ng-app="moduleName"` directive. Controllers are attached to the module to manage the `{{\$scope}}`.

HTML/CSS/JS

```

<!-- HTML View -->
<div ng-app="studentApp" ng-controller="studentCtrl">
  <h2>Student Information</h2>
  <p>Name: {{ student.name }}</p>
  <p>Course: {{ student.course }}</p>
  <p>Status: {{ getStatus() }}</p>
</div>

<!-- JavaScript Module & Controller -->
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
<script>
// 1. Define the Application Module (empty array [] = dependencies)
var app = angular.module("studentApp", []);

// 2. Attach a Controller to the Module
app.controller("studentCtrl", function($scope) {
  // Model data attached to $scope
  $scope.student = {
    name: "Saurabh Tiwari",
    course: "NIELIT O-Level M2-R5.1"
  };

  // Controller method
  $scope.getStatus = function() {
    return "Enrolled & Active";
  };
});
</script>

```

MODULE & CONTROLLER ARCHITECTURE

- `angular.module('appName', [dependencies])`: Creates a new module. The second parameter `[]` is the dependency injection array.
- `ng-app="appName"`: Binds the HTML document to the specific AngularJS module.
- `ng-controller="ctrlName"`: Designates the controller function that controls that specific DOM subsection.
- The `{{\$scope}}` Object: The glue object that connects the JavaScript Controller (business logic) to the HTML View (presentation).

In the code above: The `studentApp` module packages the app. The `studentCtrl` controller stores student data on `\$scope`. In HTML, `{ student.name }` seamlessly renders 'Saurabh Tiwari'.

- `angular.module()` creates or retrieves an AngularJS module.
- The second argument `[]` in `angular.module('app', [])` specifies module dependencies.
- `\$scope` is the execution context object bridging the Controller and the View.

QUICK TOPIC RECAP

AngularJS modules (`angular.module('app', [])`) group application components. Controllers attached to modules use `\$scope` to pass data to the HTML view.

TOPIC 6.11: ANGULAR JS DIRECTIVES (NG-APP, NG-MODEL, NG-BIND, NG-REPEAT, NG-INIT)

AngularJS Directives are special extended HTML attributes prefixed with **`ng-`** that instruct the AngularJS compiler to attach custom behavior or manipulate the DOM.

2. WHY IS IT IMPORTANT IN INDUSTRY & EXAMS?

Directives are the core innovation of AngularJS. They turn static HTML into an expressive, dynamic template language with loops, two-way data binding, and initialization.

3. HOW DOES IT WORK? (MECHANISM & DATA FLOW)

When the page loads, AngularJS parses the DOM, identifies `ng-` attributes, and executes their corresponding framework routines.

HTML/CSS/JS

```

<!-- Complete Directives Demonstration -->
<div ng-app="" ng-init="names=['HTML5', 'CSS3', 'JavaScript', 'AngularJS']">
<!-- Two-Way Data Binding with ng-model and ng-bind -->
<p>Type your name: <input type="text" ng-model="myName"></p>
<h3>Hello, <span ng-bind="myName"></span>!</h3>

<!-- Looping through array with ng-repeat -->
<h4>Syllabus Technologies:</h4>
<ul>
<li ng-repeat="tech in names">{{ tech }}</li>
</ul>
</div>

```

CORE ANGULARJS DIRECTIVES SPECIFICATION

Directive Name	Syntax Format	Functional Mechanism & Practical Role
`ng-app`	<div ng-app="myApp">	Mandatory root directive ; bootstraps and initializes an AngularJS application
`ng-model`	<input ng-model="userName">	Binds the value of HTML form controls to application data (Two-Way Data Binding)
`ng-bind`		Replaces the innerHTML of an element with the value of the specified model (alternative to `{ { } }`)
`ng-repeat`	<li ng-repeat="x in list">	Clones HTML elements for every item in an array (Looping directive like for-each)
`ng-init`	<div ng-init="price=10; qty=5">	Initializes application variables with starting default values directly inside HTML

<code>`ng-click`</code>	<code><button ng-click="count = count + 1"></code>	Attaches custom click event behavior without DOM event listeners
<code>`ng-hide` / `ng-show`</code>	<code><p ng-show="isLoggedIn"></code>	Shows or hides HTML elements conditionally based on Boolean state

When typing into `<input ng-model="myName">`: Every character you press instantly appears inside `<span ng-bind="myName"></span>` in real time with zero submit buttons or event listeners. This is two-way data binding in action.

- All core AngularJS directives are prefixed with ``ng-`` (short for Angular).
- ``ng-app`` defines the root element of an AngularJS application.
- ``ng-model`` provides two-way data binding between form inputs and the application model.
- ``ng-repeat`` repeats an HTML element for each item in a collection or array.

QUICK TOPIC RECAP

Directives (``ng-``) extend HTML: ``ng-app`` (bootstraps app), ``ng-model`` (two-way data binding), ``ng-bind`` (updates text), ``ng-repeat`` (loops arrays), ``ng-init`` (initializes variables).

TOPIC undefined:

- JavaScript (created by Brendan Eich in 1995 at Netscape) is the client-side programming language of the Web; it is completely unrelated to Java.
- Variables: ``var`` (function scope, legacy), ``let`` (block scope, reassignable), and ``const`` (block scope, immutable constant).
- Equality operators: ``==`` compares values with type coercion; ``===`` checks strict value AND data type without coercion.
- Conditional statements: ``if...else`` handles logical/range decisions; ``switch...case`` handles discrete value matching with ``break`` statements.
- Popup boxes: ``alert(msg)`` (notification, OK button), ``confirm(msg)`` (verification, returns ``true`/`false``), ``prompt(msg)`` (text input, returns string or ``null``).
- DOM events: ``onclick`` (clicks), ``onchange`` (value changes), ``onsubmit`` (form submission), ``onload`` (page ready), ``onmouseover`/`onmouseout`` (mouse hover).
- Form validation: Intercept form via ``onsubmit="return validateForm()"; return `false;`` to cancel submission if errors are found.
- AngularJS (Google, 2009) is a structural framework for Single Page Applications (SPAs) based on Model-View-Controller (MVC).
- AngularJS expressions (``{{ expression }}``) evaluate and output math, strings, and ``$scope`` data inside HTML without throwing errors.
- AngularJS modules (``angular.module('app', [])``) package applications; controllers use ``$scope`` to bridge business logic and the HTML view.
- Core directives: ``ng-app`` (root element), ``ng-model`` (two-way data binding), ``ng-bind`` (updates text), ``ng-repeat`` (loops collections), ``ng-init`` (initializes variables).

KEY TERMS, ACRONYMS & OFFICIAL FULL FORMS